

Tarea: Construcción de un Analizador con ANTLR4 y JavaScript
Tema: 25914_11

Legajo: 51923

Nombre: Ezequiel Calderon

Se proporciona una gramática en notación EBNF que describe un lenguaje específico.

Utilizando **ANTLR4** con **JavaScript**, implemente un analizador que procese un archivo de entrada (input.txt) con código fuente escrito en dicho lenguaje.

```
<programa>::= { <declaración> | <función> | <ejecución> }
<declaración>::= "variable" <nombre> [ "=" <valor> ] ";"
<función>::= "función" <nombre> ["(" <argumentos> ")"] "{" <instrucciones> "}"
<argumentos>::= <variable> ["," <argumentos>]
<instrucciones>::= { <operación_texto> | <impresión> | <retorno> }
<operación_texto> ::= <variable> "=" <transformación> "(" <cadena> ")" ";"
<transformación> ::= "mayúsculas" | "minúsculas" | "longitud" | "invertir" |
"reemplazar" <concatenar>::= <variable> "=" <cadena> "+" <cadena> ";"
<impresión> ::= "imprimir" "(" <valor> ")" ";"
<retorno>::= "devolver" <valor> ";"
<valor>::= <texto> | <número> | <variable>
<cadena>::= <texto> | <variable>
<texto>::= "\"" [caracteres] "\""
<nombre>::= [a-zA-Z_][a-zA-Z0-9_]*
```

El analizador deberá realizar las siguientes tareas:

1. **Análisis léxico y sintáctico:** realizar el análisis léxico y sintáctico sobre el código fuente e informar si la entrada es correcta o contiene errores. En caso de errores, indicar la línea en la que ocurren y la causa del problema.
2. **Tabla de lexemas-tokens:** Generar una tabla que contenga los lexemas y sus respectivos tokens reconocidos durante el análisis léxico.
3. **Árbol de análisis sintáctico:** Construir y mostrar el árbol de análisis sintáctico concreto de la entrada. Puede representarse en formato de texto.
4. **Interpretación:** Traducir el código fuente al lenguaje **JavaScript** y ejecutarlo como lo haría un intérprete básico.

El desarrollo debe entregarse cumpliendo las pautas para la entrega establecidas en el documento **Pautas de trabajo para analizador**.

Ejemplo de Código

```
variable saludo = "hola mundo";
variable enMayusculas = mayusculas(saludo);
imprimir(enMayusculas);
```

Traducción a JavaScript

```
let saludo = "hola mundo";  
let enMayusculas = saludo.toUpperCase();  
console.log(enMayusculas);
```

```
grammar Lenguaje;  
  
// — REGLAS SINTÁCTICAS —  
  
programa  
  : sentencia* EOF  
  ;  
  
sentencia  
  : declaracion  
  | funcion  
  | impresion  
  | ejecucion  
  ;  
  
// variable x;  
// variable x = valor;  
// variable x = transformacion(cadena);  
// variable x = cadena + cadena;  
declaracion  
  : VARIABLE nombre ASIGN assignable PUNTO_COMA #declConAssign  
  | VARIABLE nombre PUNTO_COMA #declSinAssign  
  ;  
  
assignable  
  : transformacion LPAREN cadena RPAREN #assignTransform  
  | cadena MAS cadena #assignConcat  
  | valor #assignValor  
  ;  
  
funcion  
  : FUNCION nombre ( LPAREN argumentos RPAREN )? LLAVE_IZQ instrucciones LLAVE_DER  
  ;  
  
argumentos  
  : nombre ( COMA nombre )*  
  ;  
  
instrucciones  
  : instruccion*  
  ;  
  
instruccion  
  : declaracion  
  | asignacion  
  | impresion  
  | retorno  
  ;
```

```

// Asignación interna a variable existente: nombre = assignable;
asignacion
    : nombre ASIGN assignable PUNTO_COMA
    ;

impresion
    : IMPRIMIR LPAREN valor RPAREN PUNTO_COMA
    ;

retorno
    : DEVOLVER valor PUNTO_COMA
    ;

// Llamada a función definida por el usuario
ejecucion
    : nombre LPAREN ( valor ( COMA valor ) * )? RPAREN PUNTO_COMA
    ;

transformacion
    : MAYUSCULAS
    | MINUSCULAS
    | LONGITUD
    | INVERTIR
    | REEMPLAZAR
    ;

valor
    : texto
    | numero
    | nombre
    ;

cadena
    : texto
    | nombre
    ;

texto : TEXTO ;
numero : NUMERO ;
nombre : IDENTIFICADOR ;

```

```
// — REGLAS LÉXICAS —

VARIABLE      : 'variable' ;
FUNCION       : 'función' | 'function' ;
MAYUSCULAS    : 'mayúsculas' | 'mayusculas' ;
MINUSCULAS    : 'minúsculas' | 'minusculas' ;
LONGITUD      : 'longitud' ;
INVERTIR      : 'invertir' ;
REEMPLAZAR    : 'reemplazar' ;
IMPRIMIR      : 'imprimir' ;
DEVOLVER      : 'devolver' ;

ASIGN         : '=' ;
PUNTO_COMA    : ';' ;
COMA          : ',' ;
LPAREN        : '(' ;
RPAREN        : ')' ;
LLAVE_IZQ     : '{' ;
LLAVE_DER     : '}' ;
MAS           : '+' ;

TEXTO         : '"' ( ~["\r\n\\] | '\\' . )* '"' ;
NUMERO        : '-'? [0-9]+ ( '.' [0-9]+ )? ;

IDENTIFICADOR : [a-zA-Z_\u00C0-\u024F] [a-zA-Z0-9_\u00C0-\u024F]* ;

WS            : [ \t\r\n]+ -> skip ;
COMENTARIO    : '//' ~[\r\n]* -> skip ;
```